36 POSTGRES — WHAT IT IS, AND HOW TO BUILD THE SAME CRUD YOU HAVE FOR MONGO

This is a NOTES-ONLY file — nothing here has been installed or created in the project yet. It's a reference for when you're ready to actually build a Postgres version of student.module.ts.

WHAT POSTGRES IS, VS MONGO

MongoDB (what this app uses today) PostgreSQL

- Document DB: JSON-like objects	- Relational DB: rows in tables
- Collection ("students")	- Table ("students")
- Document	- Row
- Field	- Column
- Schema is soft (Mongoose enforces it at the app layer, via @Prop())	- Schema is HARD — the DB itself rejects a bad insert, via CREATE TABLE column types
- _id is an ObjectId string	- id is usually an auto-incrementing integer or a uuid
- No JOINs — you either embed related data or store a reference + a second query	- JOINs are a first-class feature — related data normally lives in separate tables, combined at query time
- Query language: Mongoose methods (.find(), .findById(), etc.)	- Query language: SQL, written directly or generated by an ORM

Rule of thumb for WHEN to reach for Postgres instead of Mongo: your data is naturally tabular/relational (orders belong to customers, students enroll in courses — lots of foreign-key relationships) and you want the DB itself to guarantee data integrity (can't insert a row referencing a student that doesn't exist). Mongo suits data that's naturally document-shaped and doesn't need strict relations.

THE ORM: TypeORM (the SQL equivalent of Mongoose here)

Mongoose maps JS classes → MongoDB documents. TypeORM maps JS classes → Postgres tables, using the same decorator style you already know:

```
Mongoose (student.schema.ts)          TypeORM equivalent
-------------------------------       -------------------------------
@Schema({ timestamps: true })         @Entity('students')
export class Student {                export class Student {
  @Prop({ required: true })             @PrimaryGeneratedColumn()
  name!: string;                        id!: number;   // <- Postgres needs an explicit PK column; Mongo
gives you _id for free

  @Prop({ required: true })             @Column()
```

```
  age!: number;                          name!: string;

  @Prop({ required: true, unique: true }) @Column()
  email!: string;                           age!: number;

  @Prop()                                @Column({ unique: true })
  course!: string;                       email!: string;
}                                        @Column({ nullable: true })
                                         course?: string;

                                         @CreateDateColumn()
                                         createdAt!: Date;   // Postgres equivalent of Mongoose's
timestamps: true

                                         @UpdateDateColumn()
                                         updatedAt!: Date;
                                       }

export const StudentSchema =
  SchemaFactory.createForClass(Student);   // (TypeORM has no equivalent line — the @Entity class itself
IS the schema, read directly off the decorators)
```

CONNECTING — TypeOrmModule.forRootAsync mirrors MongooseModule.forRootAsync

app.module.ts currently has:

```
MongooseModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (configService: ConfigService) => ({
    uri: configService.getOrThrow<string>('MONGODB_URI'),
  }),
}),
```

The Postgres equivalent, same pattern, would be:

```
TypeOrmModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (configService: ConfigService) => ({
    type: 'postgres',
    url: configService.getOrThrow<string>('POSTGRES_URI'), // e.g.
postgres://user:pass@localhost:5432/mydb
    autoLoadEntities: true,   // picks up every entity registered via forFeature(), so you don't hand-
list them all here
    synchronize: true,        // DEV ONLY — auto-creates/alters tables to match your @Entity classes.
                              // NEVER true in production: it can silently drop/alter columns.
                              // Production uses migrations instead (typeorm migration:generate /
migration:run).
  }),
}),
```

New env vars needed in .env (same idea as MONGODB_URI):

```
POSTGRES_URI=postgres://postgres:yourpassword@localhost:5432/nest_crud
```

And you need an actual Postgres server to point that at — options:

- Install Postgres locally (postgresql.org) and create a DB with `createdb nest_crud` or via pgAdmin.
- Or run one instantly in Docker, no local install: docker run --name pg-dev -e POSTGRES_PASSWORD=yourpassword -p 5432:5432 -d postgres
- Or use a free hosted instance (Neon, Supabase, Railway) — same idea as MongoDB Atlas, just gives you a URI instead of setting up a local server.

PER-FEATURE REGISTRATION — mirrors MongooseModule.forFeature

// student.module.ts today:

```
MongooseModule.forFeature([{ name: Student.name, schema: StudentSchema }])
```

// TypeORM equivalent:

```
TypeOrmModule.forFeature([Student])   // just the entity class — no separate "schema" object to pair it with
```

INJECTING THE MODEL — @InjectModel vs @InjectRepository

// student.service.ts today:

```
constructor(@InjectModel(Student.name) private studentModel: Model<StudentDocument>) {}
```

// TypeORM equivalent:

```
constructor(@InjectRepository(Student) private studentRepo: Repository<Student>) {}
```

`Repository<Student>` is TypeORM's version of `Model<StudentDocument>` — the object with all the query methods on it.

CRUD METHOD-BY-METHOD, MONGOOSE VS TYPEORM VS RAW SQL

CREATE

```
Mongoose:   const doc = new this.studentModel(dto); return doc.save();
TypeORM:    const row = this.studentRepo.create(dto); return this.studentRepo.save(row);
Raw SQL:    INSERT INTO students (name, age, email, course)
            VALUES ($1, $2, $3, $4) RETURNING *;
```

READ ALL

```
Mongoose:   return this.studentModel.find().exec();
TypeORM:    return this.studentRepo.find();
Raw SQL:    SELECT * FROM students;
```

READ ONE

```
Mongoose:  return this.studentModel.findById(id).exec();
TypeORM:   return this.studentRepo.findOneBy({ id: Number(id) });
Raw SQL:   SELECT * FROM students WHERE id = $1;
```

UPDATE (partial — PATCH-style, like this app's update() already is)

```
Mongoose:  return this.studentModel.findByIdAndUpdate(id, dto, { new: true }).exec();
TypeORM:   await this.studentRepo.update(id, dto);            // TypeORM's update() does NOT return the row
           return this.studentRepo.findOneBy({ id: Number(id) }); // so you fetch it again after
           // (or: const row = await this.studentRepo.preload({ id: Number(id), ...dto }); return this.studentRepo.save(row);)
Raw SQL:   UPDATE students SET name = $1, age = $2 WHERE id = $3 RETURNING *;
           -- (only SET the columns actually present in dto — same "partial" idea as Mongoose's merge)
```

DELETE (this app's Mongo module doesn't have this yet either — same gap either DB)

```
Mongoose:  return this.studentModel.findByIdAndDelete(id).exec();
TypeORM:   return this.studentRepo.delete(id);
Raw SQL:   DELETE FROM students WHERE id = $1;
```

NOT FOUND CHECK — same pattern as student.service.ts either way:

```
const student = await this.studentRepo.findOneBy({ id: Number(id) });
if (!student) { throw new NotFoundException(`Student ${id} not found`); }
```

CONTROLLER — SAME ROUTES/VERBS AS student.controller.ts, JUST A NEW BASE PATH

```
@Controller('student-pg')
export class StudentPgController {
  constructor(private readonly studentPgService: StudentPgService) {}

  @Post('createstudent')            // POST   /student-pg/createstudent
  @Get('getallstudents')            // GET    /student-pg/getallstudents
  @Get('getstudent/:id')            // GET    /student-pg/getstudent/:id
  @Put('updatestudent/:id')         // PUT    /student-pg/updatestudent/:id
  @Patch('updatestudent/:id')       // PATCH  /student-pg/updatestudent/:id
  @Delete('deletestudent/:id')      // DELETE /student-pg/deletestudent/:id   <- new, doesn't exist on the Mongo side yet
}
```

Note: :id here is a Postgres integer (e.g. "1", "2"), not a 24-char Mongo ObjectId hex string — that's the one gotcha when testing: don't copy-paste a Mongo _id into a Postgres route.

CURL EXAMPLES (once wired up, assuming PORT=3000)

```
curl -X POST http://localhost:3000/student-pg/createstudent \
  -H "Content-Type: application/json" \
  -d "{\"name\":\"Rahul\",\"age\":20,\"email\":\"rahul@example.com\",\"course\":\"nid\"}"
```

curl http://localhost:3000/student-pg/getallstudents

curl http://localhost:3000/student-pg/getstudent/1

```
curl -X PATCH http://localhost:3000/student-pg/updatestudent/1 \
  -H "Content-Type: application/json" \
  -d "{\"age\":21}"

curl -X PUT http://localhost:3000/student-pg/updatestudent/1 \
  -H "Content-Type: application/json" \
  -d "{\"name\":\"Rahul\",\"age\":21,\"email\":\"rahul@example.com\",\"course\":\"nid\"}"
```

curl -X DELETE http://localhost:3000/student-pg/deletestudent/1

WHAT YOU'D ACTUALLY NEED TO DO, END TO END (checklist for later)

1. Get a Postgres server reachable (local install, Docker, or hosted) — see options above.
2. npm install @nestjs/typeorm typeorm pg
3. Add POSTGRES_URI to .env
4. Add TypeOrmModule.forRootAsync(...) to app.module.ts imports (alongside the existing MongooseModule one — both DBs can run side by side)
5. Create src/student-pg/student.entity.ts (the @Entity class above)
6. Create src/student-pg/dto/create-student.dto.ts and update-student.dto.ts (identical to the Mongo ones — class-validator doesn't care which DB it feeds)
7. Create src/student-pg/student-pg.service.ts (Repository-based methods above)
8. Create src/student-pg/student-pg.controller.ts (routes above)
9. Create src/student-pg/student-pg.module.ts with TypeOrmModule.forFeature([Student]), import it into AppModule
10. Run the app — with synchronize: true, TypeORM creates the `students` table automatically on first boot, no manual CREATE TABLE needed in dev.

See also: **theory/30-mongodb-connection.txt**, **theory/31-mongoose-schema-decorators.txt**, **theory/32-injecting-mongoose-models.txt**, **theory/35-put-vs-patch.txt**, student.schema.ts, student.service.ts, student.controller.ts, student.module.ts